SOUGATA SAHA
IT-A2-037 OOP ASSIGNMENT 2

ASSIGNMENT-2

16. Write a simple class that represents a class of geometrical points each of which has three coordinates. The class should have appropriate constructor(s). Also add a member function distance() that calculates Euclidian distance between two points. Now create two points, find the distance between them and print it.

Soln->

```
#include <iostream>
#include <cmath>
using namespace std;
class Point3D {
private:
    double x, y, z;
public:
    Point3D(double x_, double y_, double z_) : x(x_), y(y_), z(z_) {}
    double distance(const Point3D& other) const {
        return sqrt(pow(x - other.x, 2) + pow(y - other.y, 2) + pow(z - other.z, 2));
    }
};
int main() {
    Point3D p1(1.0, 2.0, 3.0);
    Point3D p2(4.0, 6.0, 8.0);
    cout << "Distance between points: " << p1.distance(p2) << endl;
    return 0;
}
```

---

17. Write a class for the geometrical shape rectangle. Write suitable constructors and member functions. Add a member function area() that calculates the area of a rectangle. Create 4 rectangles and print their respective area.

Soln->

```
#include <iostream>
using namespace std;

class Rectangle {
private:
   double length, width;

public:
   Rectangle(double l, double w) : length(l), width(w) {}

      double area() const {
      return length * width;
   }
};
int main() {
   Rectangle r1(4, 5), r2(6, 7), r3(3.5, 2.1), r4(10, 20);
   cout << "Area of Rectangle 1: " << r1.area() << endl;
   cout << "Area of Rectangle 2: " << r2.area() << endl;
   cout << "Area of Rectangle 3: " << r3.area() << endl;
   cout << "Area of Rectangle 4: " << r4.area() << endl;

   return 0;
}
```

---

18. Write a class that represents a class of wireless device. A device has a location (point object may be used), a fixed unique id, and a fixed circular transmission range. Write suitable constructors and member functions for this class. Instantiates 10 such devices. Choose location (coordinates) and transmission range of the devices randomly. Now, for each of these devices, find the neighbor devices (i.e. devices that belong to the transmission range). Suppose, all of these devices have moved to a new location (randomly chosen). Find out the new set of neighbors for each of these devices.

```
#include <iostream>
```

```
#include <vector>
#include <cmath>
#include <cstdlib>
#include <ctime>
using namespace std;

class Point3D {
public:
    double x, y, z;

    Point3D(double x_, double y_, double z_) : x(x_), y(y_), z(z_) {}

    double distance(const Point3D& other) const {
        return sqrt(pow(x - other.x, 2) + pow(y - other.y, 2) + pow(z - other.z, 2));
    }
};

class WirelessDevice {
private:
    int id;
    Point3D location;
    double range;

public:
        WirelessDevice(int id_, double x, double y, double z, double range_)
        : id(id_), location(x, y, z), range(range_) {}

    Point3D getLocation() const { return location; }
    vector<int> findNeighbors(const vector<WirelessDevice>& devices) const {
        vector<int> neighbors;
```

```
        for (const auto& device : devices) {
            if (device.id != id &&
                location.distance(device.location) <= range) {
                neighbors.push_back(device.id);
            }
        }
        return neighbors;
    }
    void moveTo(double x, double y, double z) {
        location = Point3D(x, y, z);
    }
};

int main() {
    srand(time(0));
    vector<WirelessDevice> devices;
    for (int i = 0; i < 10; ++i) {
        double x = rand() % 100;
        double y = rand() % 100;
        double z = rand() % 100;
        double range = (rand() % 50) + 10;
        devices.emplace_back(i, x, y, z, range);
    }
    for (const auto& device : devices) {
        auto neighbors = device.findNeighbors(devices);
        cout << "Device " << device.getLocation().x << " neighbors: ";
        for (int id : neighbors) {
            cout << id << " ";
        }
        cout << endl;
```

```
    }
    for (auto& device : devices) {
        double x = rand() % 100;
        double y = rand() % 100;
        double z = rand() % 100;
        device.moveTo(x, y, z);
    }

    for (const auto& device : devices) {
        auto neighbors = device.findNeighbors(devices);
        cout << "New neighbors: ";
        for (int id : neighbors) {
            cout << id << " ";
        }
        cout << endl;
    }

    return 0;
}
```

19.Write a class Vector for one dimensional array. Write suitable constructor/copy constructor. Also add member functions for perform basic operations (such as addition, subtraction, equality, less, greater etc.). Create vectors and check if those operations are working correctly.

Soln->

```
#include <iostream>
#include <vector>
using namespace std;

class Vector {
private:
```

```
    vector<int> data;

public:
    Vector(const vector<int>& values) : data(values) {}
    Vector operator+(const Vector& other) const {
        vector<int> result(data.size());
        for (size_t i = 0; i < data.size(); ++i) {
            result[i] = data[i] + other.data[i];
        }
        return Vector(result);
    }
    void print() const {
        for (int x : data) {
            cout << x << " ";
        }
        cout << endl;
    }
};

int main() {
    Vector v1({1, 2, 3});
    Vector v2({4, 5, 6});

    Vector sum = v1 + v2;

    cout << "Sum of vectors: ";
    sum.print();

    return 0;
}
```

20. Write a class IntArray for one dimensional integer array. Implement the necessary constructor, copy constructor, and destructor (if necessary) in this class. Implement other member functions to perform operations, such adding two arrays, reversing an array, sorting an array etc. Create an IntArray object having elements 1, 2 and 3 in it. Print its elements. Now, create another IntArray object which is an exact copy of the previous object. Print its elements. Now, reverse the elements of the last object. Finally print elements of both the objects.

Soln->

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class IntArray {
private:
    vector<int> data;

public:
    IntArray(const vector<int>& values) : data(values) {}
    IntArray(const IntArray& other) : data(other.data) {}

        void reverseArray() {
        reverse(data.begin(), data.end());
    }
    void print() const {
        for (int x : data) {
            cout << x << " ";
        }
        cout << endl;
    }
};
```

```
int main() {
    IntArray arr1({1, 2, 3});
    cout << "Original Array: ";
    arr1.print();

    IntArray arr2 = arr1; // Copy array
    cout << "Copied Array: ";
    arr2.print();

    arr2.reverseArray();
    cout << "Reversed Copied Array: ";
    arr2.print();

    cout << "Original Array After Reversing Copied Array: ";
    arr1.print();

    return 0;
}
```

---

21. Create a simple class SavingsAccount for savings account used in banks as follows: Each SavingsAccount object should have three data members to store the account holder's name, unique account number and balance of the account. Assume account numbers are integers and generated sequentially. Note that once an account number is allocated to an account, it does not change during the entire operational period of the account. The bank also specifies a rate of interest for all savings accounts created. Write relevant methods (such as withdraw, deposit etc.) in the class. The bank restricts that each account must have a minimum balance of Rs. 1000. Now create 100 SavingsAccount objects

specifying balance at random ranging from Rs. 1,000 to 1,00,000. Now, calculate the interest for one year to be paid to each account and deposit the interest to the corresponding balance. Also find out total amount of interest to be paid to all accounts in one year.

Soln->

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

class SavingsAccount {
private:
    static int accountNumberCounter;
    string holderName;
    int accountNumber;
    double balance;
    static constexpr double interestRate = 0.04; // 4% interest rate
    static constexpr double minBalance = 1000.0;

public:
    SavingsAccount(string name, double initialBalance)
        : holderName(name), accountNumber(accountNumberCounter++), balance(initialBalance) {}

    void deposit(double amount) { balance += amount; }
    bool withdraw(double amount) {
        if (balance - amount >= minBalance) {
            balance -= amount;
            return true;
        }
```

```
        return false;
    }
    void addInterest() { balance += balance * interestRate; }
    double getBalance() const { return balance; }
    static int getTotalAccounts() { return accountNumberCounter - 1; }
};

int SavingsAccount::accountNumberCounter = 1;

int main() {
    srand(time(0));
    vector<SavingsAccount> accounts;

    for (int i = 0; i < 100; ++i) {
        double balance = 1000 + rand() % 99001;
        accounts.emplace_back("Holder" + to_string(i + 1), balance);
    }

    double totalInterest = 0.0;
    for (auto& account : accounts) {
        double oldBalance = account.getBalance();
        account.addInterest();
        totalInterest += account.getBalance() - oldBalance;
    }

    cout << "Total interest paid to all accounts: " << totalInterest << endl;
    return 0;
}
```

---

22. Write some programs to understand the notion of constant member functions, mutable data members etc.

Soln->

```
#include <iostream>
using namespace std;

class Example {
private:
    mutable int accessCount;
    int value;

public:
    Example(int val) : value(val), accessCount(0) {}

    int getValue() const {
        accessCount++;
        return value;
    }

    void setValue(int val) { value = val; }

    int getAccessCount() const { return accessCount; }
};

int main() {
    Example ex(10);
    cout << "Value: " << ex.getValue() << endl;
    cout << "Access count: " << ex.getAccessCount() << endl;
    ex.setValue(20);
    cout << "Value: " << ex.getValue() << endl;
```

```
    cout << "Access count: " << ex.getAccessCount() << endl;
    return 0;
}
```

---

23. Write the definition for a class called Complex that has private floating point data members for storing

real and imaginary parts. The class has the following public member functions:

setReal() and setImg() to set the real and imaginary part respectively.

getReal() and getImg() to get the real and imaginary part respectively.

disp() to display complex number object

sum() to sum two complex numbers & return a complex number

Write main function to create three complex number objects. Set the value in two objects and call sum()

to calculate sum and assign it in third object. Display all complex numbers.

Soln->

```
#include <iostream>
using namespace std;

class Complex {
private:
    float real, img;

public:
    void setReal(float r) { real = r; }
    void setImg(float i) { img = i; }
    float getReal() const { return real; }
    float getImg() const { return img; }
    void disp() const { cout << real << " + " << img << "i" << endl; }
    Complex sum(const Complex& other) const {
        Complex result;
        result.setReal(real + other.real);
```

```
        result.setImg(img + other.img);
        return result;
    }
};

int main() {
    Complex c1, c2, c3;
    c1.setReal(2.5);
    c1.setImg(3.5);
    c2.setReal(1.5);
    c2.setImg(4.0);
    c3 = c1.sum(c2);
    cout << "First Complex: ";
    c1.disp();
    cout << "Second Complex: ";
    c2.disp();
    cout << "Sum: ";
    c3.disp();
    return 0;
}
```

24. Complete the class with all function definitions for a stack

```
class Stack {
int *buffer, top;
public :
Stack(int); //create a stack with specified size
void push(int); //push the specified item
int pop(); //return the top element
void disp(); //displays elements in the stack in top to bottom order
```

};

Now, create a stack with size 10, push 2, 3, 4 and 5 in that order and finally pop one element. Display

elements present in the stack.

Soln->

```
#include <iostream>
using namespace std;

class Stack {
private:
    int* buffer;
    int top, size;

public:
    Stack(int s) : size(s), top(-1) { buffer = new int[s]; }
    ~Stack() { delete[] buffer; }
    void push(int item) { if (top < size - 1) buffer[++top] = item; }
    int pop() { return (top >= 0) ? buffer[top--] : -1; }
    void disp() {
        for (int i = top; i >= 0; --i) cout << buffer[i] << " ";
        cout << endl;
    }
};

int main() {
    Stack s(10);
    s.push(2);
    s.push(3);
    s.push(4);
    s.push(5);
```

```
    s.pop();
    s.disp();
    return 0;
}
```

---

25. Complete the class with all function definitions for a circular queue

class Queue {

int *data;

int front, rear;

public :

Queue(int); //create queue with specified size

void add(int);//add specified element to the queue

int remove();//delete element from the queue

void disp(); //displays all elements in the queue(front to rear order)

};

Now, create a queue with size 10 add 2, 3, 4 and 5 in that order and finally delete two elements. Display

elements present in the stack.

Soln->

```
#include <iostream>
using namespace std;

class Queue {
private:
    int* data;
    int front, rear, size, capacity;

public:
```

```
    Queue(int c) : capacity(c), front(-1), rear(-1), size(0) {
        data = new int[c];
    }
    ~Queue() { delete[] data; }
    void add(int item) {
        if (size < capacity) {
            rear = (rear + 1) % capacity;
            data[rear] = item;
            if (front == -1) front = 0;
            size++;
        }
    }
    int remove() {
        if (size > 0) {
            int temp = data[front];
            front = (front + 1) % capacity;
            size--;
            return temp;
        }
        return -1;
    }
    void disp() {
        for (int i = 0; i < size; ++i)
            cout << data[(front + i) % capacity] << " ";
        cout << endl;
    }
};

int main() {
    Queue q(10);
```

```
    q.add(2);
    q.add(3);
    q.add(4);
    q.add(5);
    q.remove();
    q.remove();
    q.disp();
    return 0;
}
```

---

26. Write a class for your Grade card. The grade card is given to each student of a department per semester.

The grade card typically contains the name of the department, name of the student, roll number,

semester, a list of subjects with their marks and a calculated CGPA. Create 60 such grade cards in a 3rd

semester with relevant data and find the name and roll number of student having highest CGPA.

Soln->

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>
using namespace std;

class GradeCard {
private:
    string departmentName;
    string studentName;
    int rollNumber;
    int semester;
```

```
    vector<pair<string, int>> subjects; // Subject name and marks
    double cgpa;

    void calculateCGPA() {
        int totalMarks = 0;
        for (const auto& subject : subjects) totalMarks += subject.second;
        cgpa = totalMarks / (double)(subjects.size() * 10);
    }

public:
    GradeCard(string dept, string name, int roll, int sem, const vector<pair<string, int>>& subs)
        : departmentName(dept), studentName(name), rollNumber(roll), semester(sem), subjects(subs) {
        calculateCGPA();
    }

    double getCGPA() const { return cgpa; }
    string getStudentName() const { return studentName; }
    int getRollNumber() const { return rollNumber; }
};

int main() {
    srand(time(0));
    vector<GradeCard> gradeCards;
    vector<pair<string, int>> subjects = {{"Math", 0}, {"Physics", 0}, {"Chemistry", 0}, {"Biology", 0}};

    for (int i = 0; i < 60; ++i) {
        for (auto& subject : subjects) subject.second = 40 + rand() % 61; // Marks between 40 and 100
        gradeCards.emplace_back("Computer Science", "Student" + to_string(i + 1), i + 1, 3, subjects);
    }
```

```
    double maxCGPA = 0.0;
    string topStudent;
    int topRollNumber = 0;

    for (const auto& card : gradeCards) {
        if (card.getCGPA() > maxCGPA) {
            maxCGPA = card.getCGPA();
            topStudent = card.getStudentName();
            topRollNumber = card.getRollNumber();
        }
    }

    cout << "Top Student: " << topStudent << ", Roll Number: " << topRollNumber << ", CGPA: " << maxCGPA << endl;
    return 0;
}
```

---

27. Create a class for Book. A book has unique isbn (string), title, a list of authors, and a price. Write relevant functions. Now write a class BookStore which has a list of books. There may be multiple copies of a book in the book store. Write relevant member functions. Write a function books() that returns list of unique isbn numbers of the books, a function noOfCopies() that returns the number of copies available for a given isbn number and a function totalPrice() that returns the total price of all the books. Create a book store having a number of books (multiple copies). Now, for each book, print number of copies of that book along with its title.

Soln->

```
#include <iostream>
```

```
#include <vector>
#include <string>
#include <unordered_map>
using namespace std;

class Book {
private:
    string isbn;
    string title;
    vector<string> authors;
    double price;

public:
    Book(string i, string t, const vector<string>& a, double p) : isbn(i), title(t), authors(a), price(p) {}

    string getISBN() const { return isbn; }
    string getTitle() const { return title; }
    double getPrice() const { return price; }
};

class BookStore {
private:
    unordered_map<string, pair<Book, int>> inventory; // ISBN -> (Book object, count)

public:
    void addBook(const Book& book, int count) {
        inventory[book.getISBN()] = {book, count};
    }

    vector<string> books() const {
```

```
        vector<string> isbnList;
        for (const auto& entry : inventory) isbnList.push_back(entry.first);
        return isbnList;
    }

    int noOfCopies(const string& isbn) const {
        if (inventory.find(isbn) != inventory.end()) return inventory.at(isbn).second;
        return 0;
    }

    double totalPrice() const {
        double total = 0.0;
        for (const auto& entry : inventory) total += entry.second.first.getPrice() * entry.second.second;
        return total;
    }
};

int main() {
    BookStore store;
    store.addBook(Book("123", "Book A", {"Author1", "Author2"}, 250.0), 5);
    store.addBook(Book("456", "Book B", {"Author3"}, 300.0), 3);
    store.addBook(Book("789", "Book C", {"Author4", "Author5"}, 150.0), 7);

    for (const auto& isbn : store.books()) {
        cout << "ISBN: " << isbn << ", Title: " << store.noOfCopies(isbn) << " copies available" << endl;
    }

    cout << "Total Price of all books: " << store.totalPrice() << endl;
    return 0;
}
```